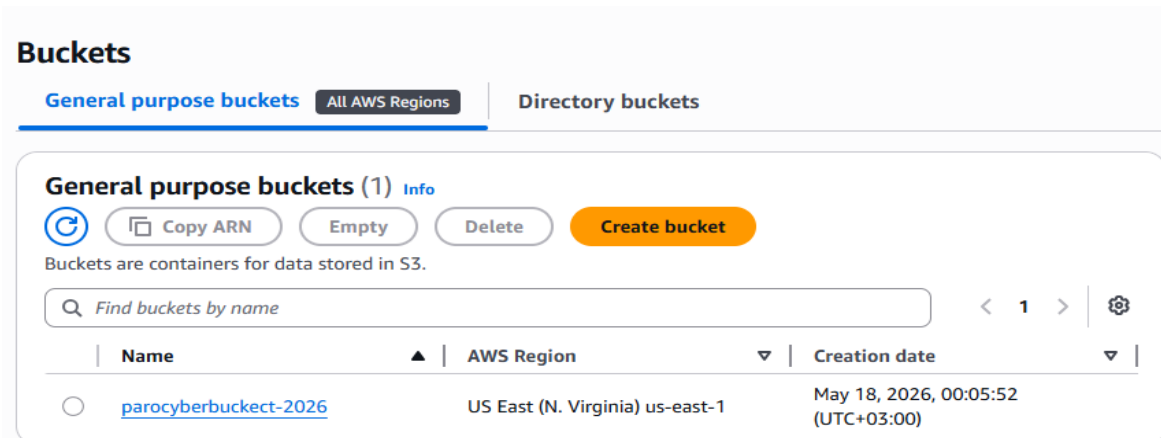


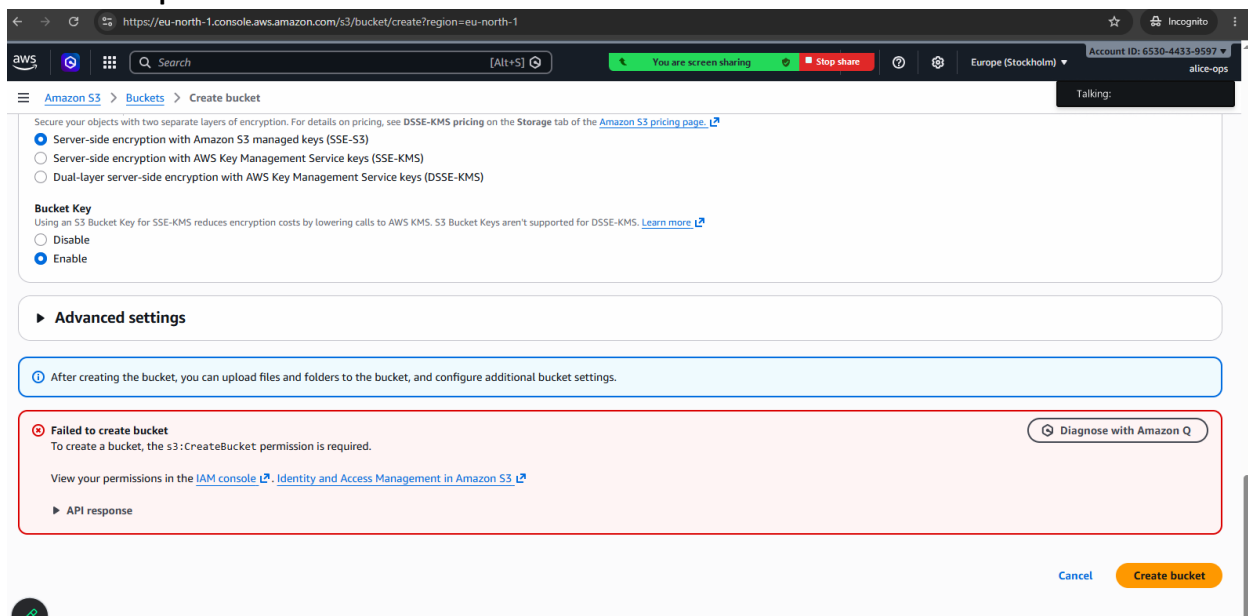
LAB 1 -- IAM: Users, Groups, Effective Permissions, and Permission Boundaries

Step 4 Observation Checkpoint — alice-ops S3 Console Access



Alice can see the existing bucket parocyberbucket-2026 (created May 18, 2026). This confirms her AmazonS3ReadOnlyAccess group policy is working — she can list and view buckets.

User: alice-ops S3 Create Bucket — Failed to create bucket



Alice attempts to create an S3 bucket. The error reads: "Failed to create bucket — To create a bucket, the s3:CreateBucket permission is required." This confirms that despite the inline policy granting s3:PutObject and s3>DeleteObject, s3:CreateBucket was not included, so bucket creation fails.

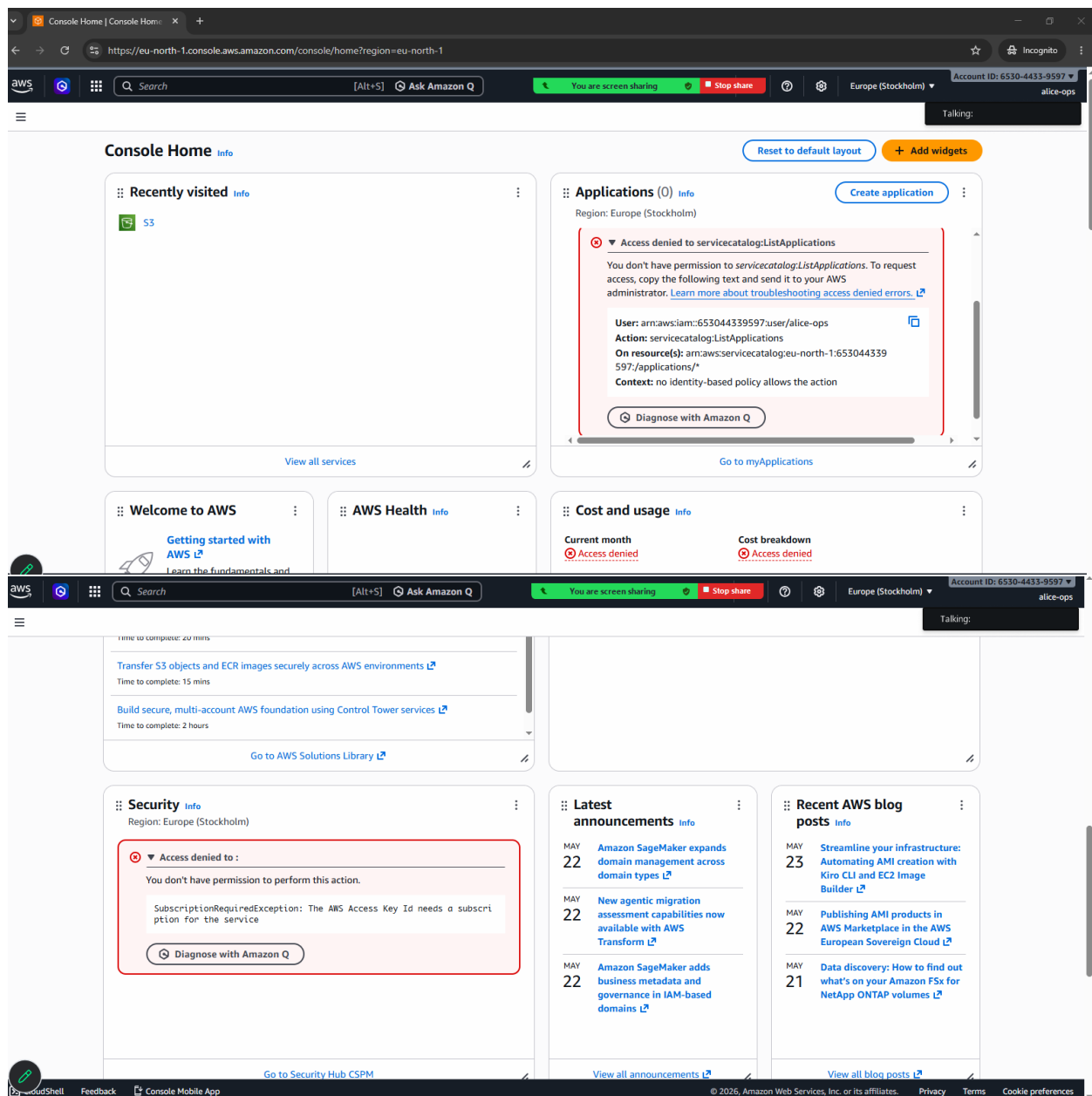
Observation Checkpoint -- After Step 4

1. Could alice-ops delete an object? If yes, why? If no, why not?

- **Yes, initially.** She could delete *objects* (files) because her personal inline policy explicitly granted her `s3:DeleteObject`.
- **However, she could NOT delete the bucket itself.** Deleting the bucket failed because her inline policy only granted object-level management (`s3:DeleteObject`, `s3:PutObject`, `s3:GetObject`), *not* bucket management privileges (`s3:DeleteBucket`).

2. Did anything surprise you about what was available or not available in the console?

- The S3 Console still displays buttons for actions like "Delete bucket" or "Create bucket" even if you don't have the permissions to use them. The console doesn't dynamically hide these administrative features; it only evaluates whether you are actually authorized *after* you click the button and submit the request, resulting in a red error banner.



This shows alice-ops signed into the AWS Console. The Applications widget displays an "Access denied" error for servicecatalog:ListApplications, confirming alice-ops has limited permissions. Context field reads: *"no identity-based policy allows the action"* expected behavior for a user whose policies don't include Service Catalog access.

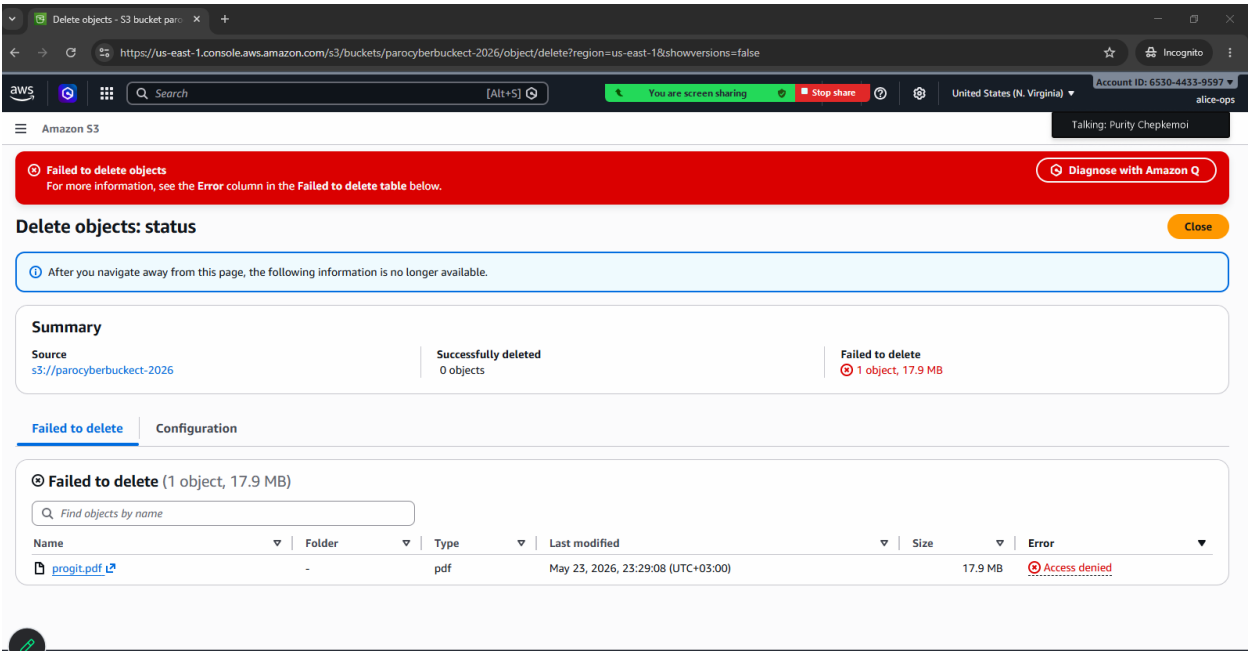
3. The group has AmazonS3ReadOnlyAccess. That policy does not explicitly deny anything... Was your prediction correct?

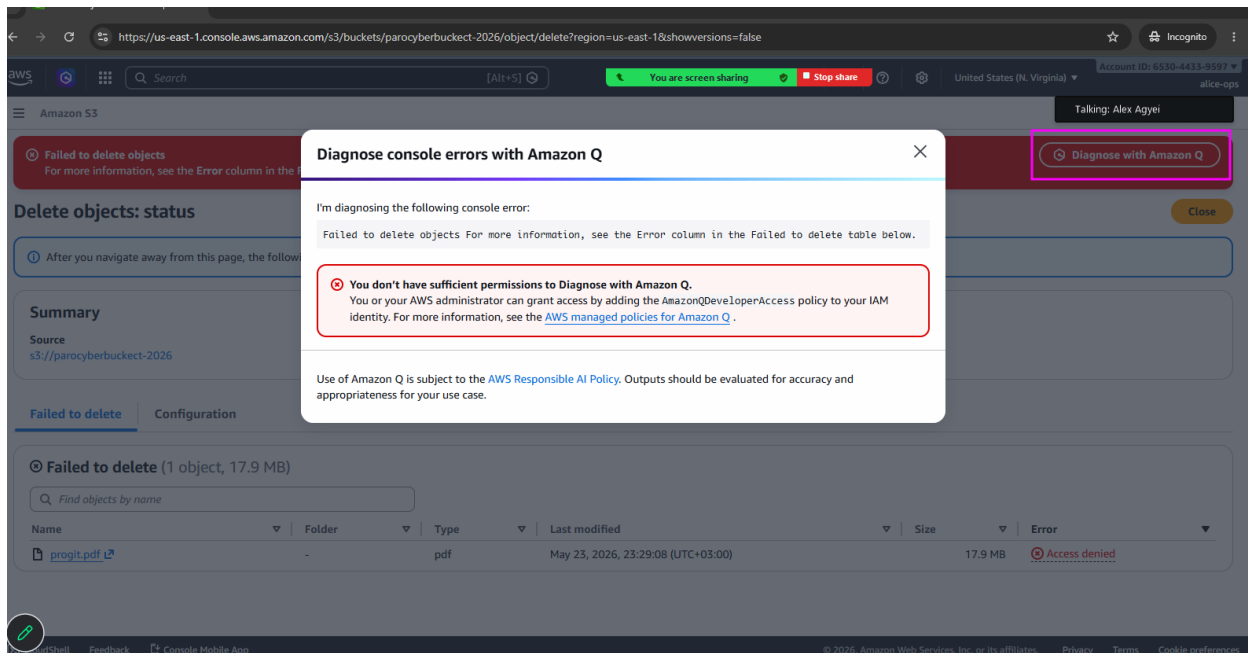
- **Yes.** In AWS, an implicit deny (the absence of an allow) is easily overridden by an explicit Allow statement elsewhere. Because the group policy simply omitted write permissions rather than explicitly blocking them, Alice's personal inline policy successfully expanded her capabilities to include object creation and deletion.

Step 5 Observation Checkpoint — After Adding Group Deny for S3 Delete

S3 Delete Objects: Status — Access denied on object deletion

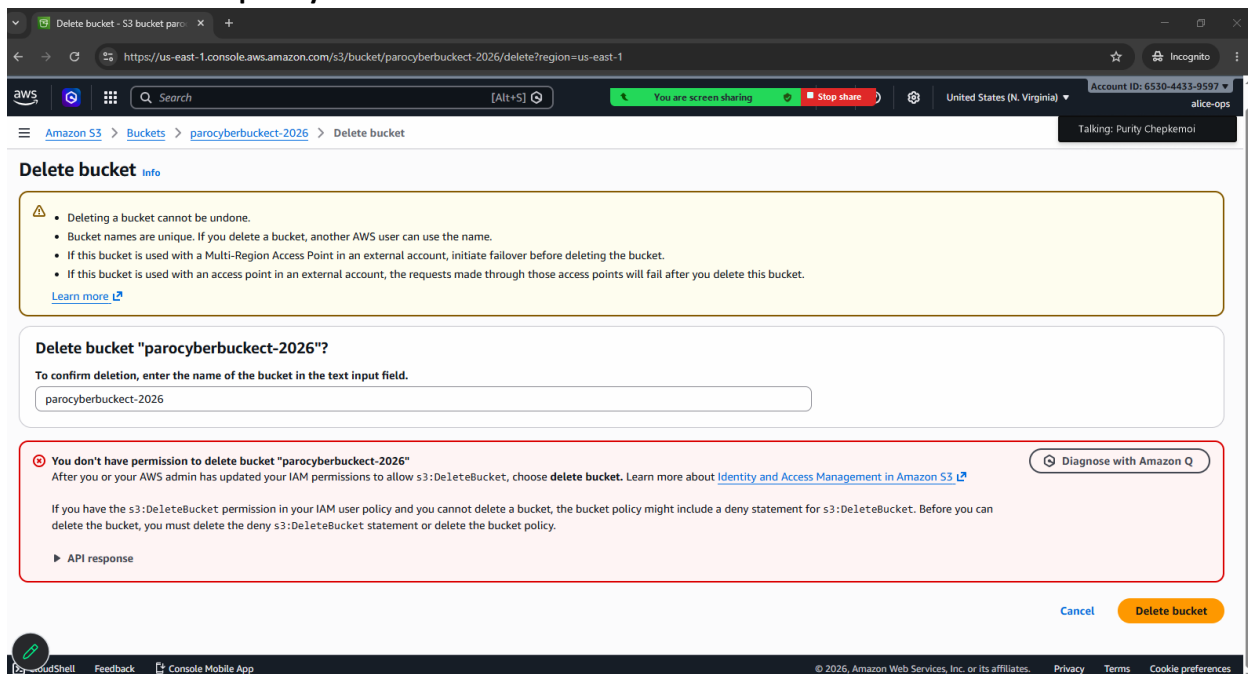
After the cloudops-deny-s3-delete inline policy is added to CloudOpsTeam, alice-ops attempts to delete progit.pdf from parocyberbucket-2026. Result: **0 objects successfully deleted, 1 object failed** — Error column shows **"Access denied."** This is the key demonstration that the explicit group Deny overrides alice's personal Allow policy.





Clicking the "Diagnose with Amazon Q" after the failed object deletion. The dialog confirms the error: "Failed to delete objects." A secondary error notes `alice-ops` lacks `AmazonQDeveloperAccess` to use the diagnostic tool — both errors visible simultaneously, useful for demonstrating layered permission restrictions.

S3 Delete Bucket "parocyberbucket-2026" — Permission denied



Alice attempts to delete the bucket itself. Error: "You don't have permission to delete bucket 'parocyberbucket-2026'." The console notes that the IAM deny statement for `s3:DeleteBucket` is blocking the action — direct confirmation that the group deny policy is in effect.

Observation Checkpoint -- After Step 5

1. Before this step, could alice-ops delete objects? After adding the group deny, can she?

- **Before:** Yes, her inline policy allowed it.
- **After:** No, she cannot. The explicit Deny policy (cloudops-deny-s3-delete) was attached to her group, her object deletion privileges were instantly blocked with an "Access Denied" error.

2. Explain in your own words why these two situations are different and why the result changed.

- In the first situation, the group simply remained silent about deleting files (an implicit deny), which allowed Alice's personal policy to speak up and grant the permission. In the second situation, the group actively shouts "No" (an explicit deny). In AWS, a targeted, active rejection completely silences any other permission source.

3. What is the AWS rule that governs this behavior? Write it in one sentence.

- An explicit Deny in any applicable policy always overrides an Allow, no matter where that allow comes from.

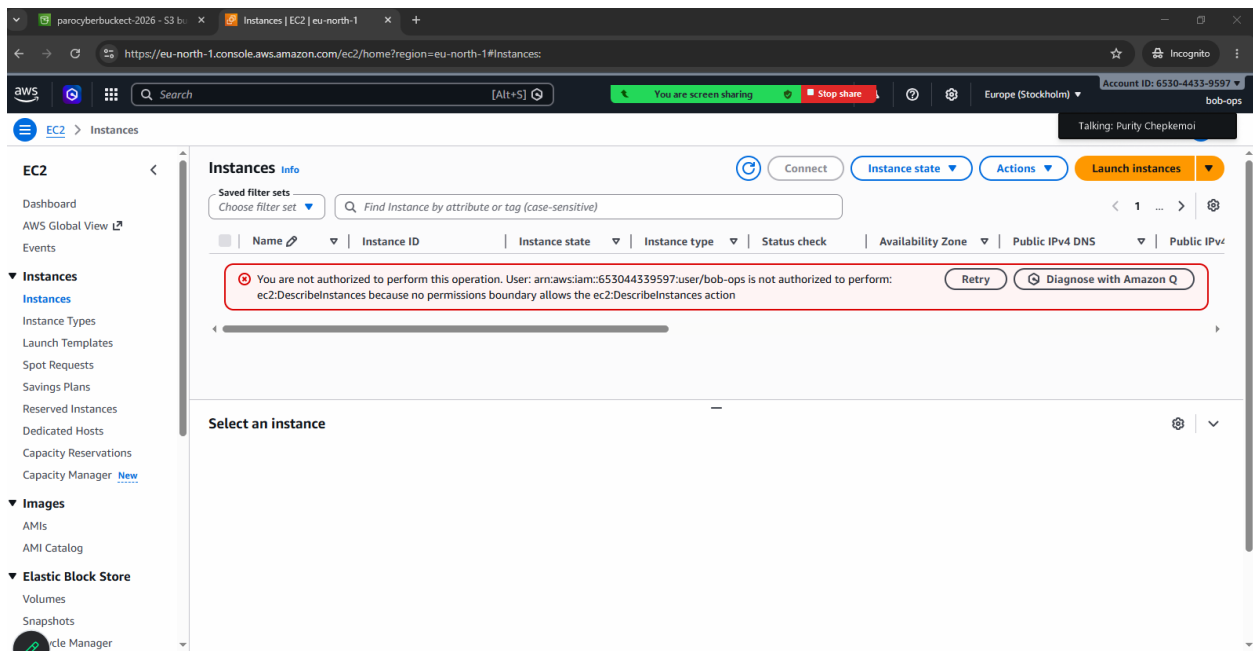
4. Can you think of a real-world scenario where you would want an explicit deny rather than simply not granting a permission?

- You would use an explicit deny during a security incident or corporate offboarding. For example, if an engineer is suspected of data exfiltration, you can attach a global Deny to their user group to immediately strip all access across the board, guaranteeing they cannot touch production data even if they have highly privileged personal or role-based policies attached elsewhere.

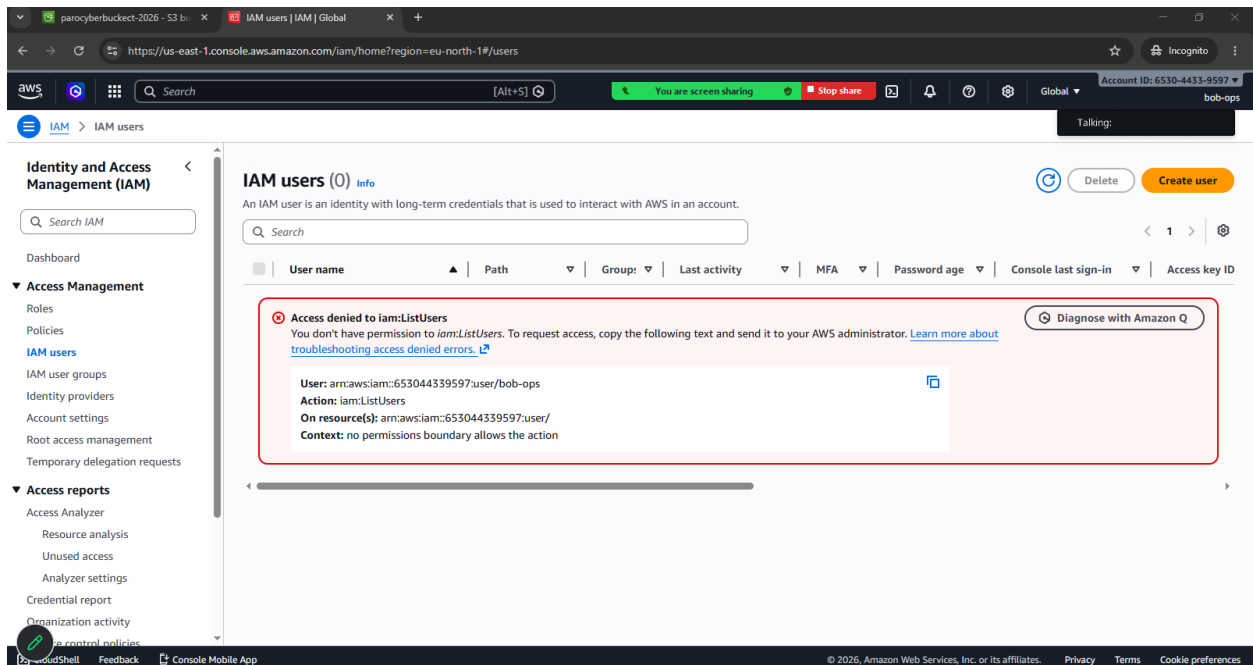
Step 6 Observation Checkpoint — bob-ops with Permission Boundary

EC2 → Instances — Unauthorized: no permissions boundary allows ec2:DescribeInstances

bob-ops navigates to EC2 Instances. Error: *"You are not authorized to perform this operation... bob-ops is not authorized to perform ec2:DescribeInstances because no permissions boundary allows the ec2:DescribeInstances action."* This is the critical boundary demonstration — bob has Action: * in his identity policy but the boundary (boundary-s3-readonly-only) blocks all non-S3 actions.



IAM Users — Access denied to iam:ListUsers



bob-ops attempts to view IAM users. Error: "Access denied to iam:ListUsers... Context: no permissions boundary allows the action." Confirms the boundary blocks IAM actions entirely, even though bob's identity policy technically allows everything.

Observation Checkpoint -- After Step 6

1. Bob has Action: * in his identity policy. That literally means every action. Why can he not create an EC2 instance?

- Because of his **Permission Boundary**. A permission boundary sets the absolute maximum ceiling for an identity. Since Bob's boundary policy (boundary-s3-readonly-only) only whitelists S3 read actions, any service or operation outside of S3 (like ec2:DescribeInstances or iam:ListUsers) is strictly blocked, completely neutralizing his administrator-level identity policy.

2. How does a permission boundary differ from a group deny? Both are blocking Bob from certain actions. Are they the same mechanism?

- **No, they are different mechanisms.** A group deny is an active, targeted blockade against specific actions. A permission boundary is a filter or a "ceiling." A group deny says *"You can never do this specific thing."* A permission boundary says *"You can only do what fits inside this box, and everything outside of it is structurally ignored."*

3. Write the formula for how effective permissions work when a boundary is present. Use plain language, not just the math notation.

- An action is only allowed if it is explicitly permitted by **both** the user's identity policy (or group policies) **AND** the attached permission boundary. If it's missing from either one, it is denied.

4. If you removed the boundary tomorrow but left the bob-admin-attempt policy, what would bob-ops be able to do?

Bob would instantly become a full **Cloud Administrator**. Without the boundary ceiling filtering his requests, his Action: "*" inline policy would give him unrestricted power to create, modify, and delete any resource, user, or billing metric across the entire AWS account.

Step 7: Observe charlie-ops -- No Personal Policies

Instance launch failed

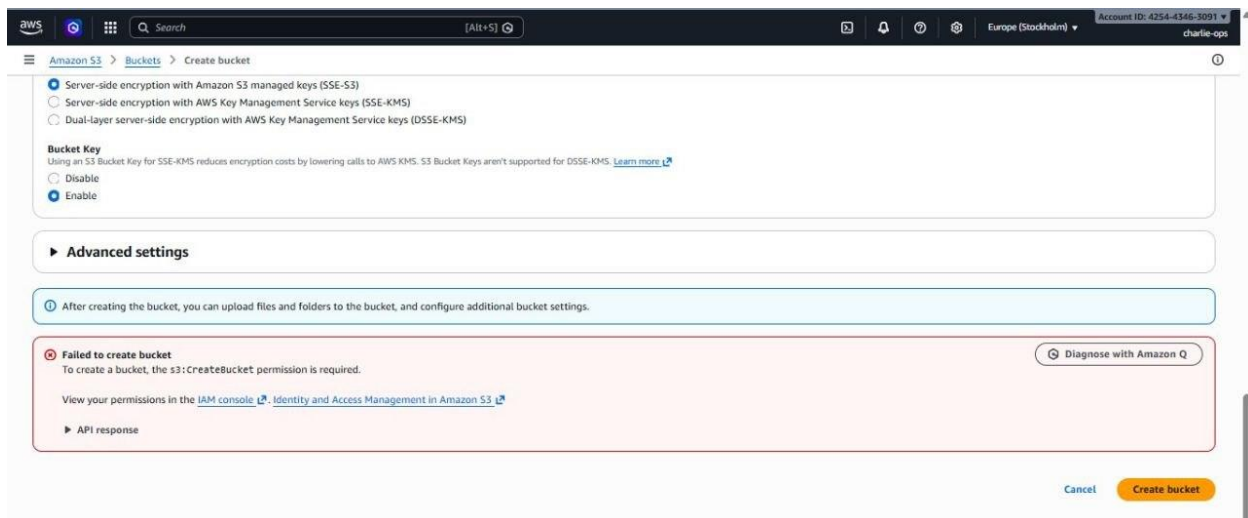
You are not authorized to perform this operation. User: arn:aws:iam::425443463091:user/charlie-ops is not authorized to perform: ec2:CreateSecurityGroup on resource: arn:aws:ec2:eu-north-1:425443463091:vpce/vpc-05bf6c711d6e28b2e because no identity-based policy allows the ec2:CreateSecurityGroup action. Encoded authorization failure message: rh0ZycqAFcScNxy80gynZuQvUjeGeoReSU05mDEcyPeCucyEdrlQmqgAzjp3HutdgvhgKY2MPLzTN6Aazw1ybvbrh2JVMEwG6e7x8DzPP3LTqOd6AQwmrU9BLWEVPZr2R0pfq80TGzjKCZ-HSke-qBEC_gQld01GCx5dmd-XSHH2L8wYylCEIZhcbYOWsfJCZlowgOxXknnZWz2nw7bCZAIGW1NBW4QnHtkYVZ667Kkpid2F1TQHlopcKNYzXT5fd2tUoZjX0zizQ15o9-lcMj5vHfR9gg430024gYqJ_DUVcqkh2d2jXYMgFD4Ukg2gg4LNVv2DhTCT7fpQeymX4VQqAVUdeW_H2pW_EsRMauqLDhandVR0K6ZWNFHLfu_ECHz2dBRRTUkVv8LMykOH27u3CCZDI-BfVAERgarj8ITU4hrJedkiYmQGokqFGvaktOmB0wR2rvcC9ACDUPsqazDH4u_rYV_YYSHVod-pDYngalGrJaeSj5pWZnGgy_cME_uRvqrmthY8BR0x-xAp7A-ddyDOQmjfcC9q2HecrGXTCw4Bfh

[Diagnose with Amazon Q](#)

Launch log

Task	Status
Initializing requests	Succeeded
Creating security groups	Failed

[Cancel](#) [Edit instance config](#) [Retry failed tasks](#)



- Charlie-ops possesses no inline adjustments, meaning his access profile is purely defined by the standard group inheritance rules. He can easily read and describe general EC2 attributes and list S3 contents, but he cannot delete objects or change configurations. This serves as the clean baseline control state for the CloudOpsTeam profile.

OBSERVATIONS

- **For Alice (Steps 4 & 5):** Alice successfully read resources due to the group policy and initially could delete objects because her inline policy expanded her permissions. However, the moment the explicit Deny was attached to CloudOpsTeam, it evaluated first and obliterated her personal Allow. She could not delete buckets because s3:DeleteBucket was missing from her personal inline policy anyway.
- **For Bob (Step 6):** An identity policy with Action: "*" is entirely useless outside of S3 reading if a Permission Boundary blocks it.
- Because services like EC2 and IAM were not explicitly whitelisted in Bob's boundary, they were immediately denied, despite his admin identity policy.
- An explicit Deny *always* overrides an Allow, and a Permission Boundary defines the absolute maximum permission boundary, acting as a ceiling that identity policies cannot break through.

Final Reflection Questions

1. If I attach a policy that allows EC2 full access directly to alice-ops, will she be able to launch EC2 instances? Walk through your reasoning using the policy evaluation logic.

Yes, she will be able to launch EC2 instances. Reasoning:

- **No Explicit Deny:** According to the documentation, AWS policy evaluation starts with a default deny. We must look for an explicit Deny first. While her group (CloudOpsTeam) has an explicit deny policy attached (cloudops-deny-s3-delete), that policy is strictly targeted at blocking S3 deletions (s3:DeleteObject). It does not contain an explicit deny for EC2 actions.

- **Overriding the Implicit Deny:** Initially, Alice cannot launch EC2 instances because her group only inherits AmazonS3ReadOnlyAccess, leaving an implicit deny (absence of an allow) for EC2.
- **The IAM Evaluation Rule:** When you attach an identity-based policy directly to Alice allowing EC2 full access, it provides the necessary explicit Allow statement. Because an explicit Allow easily overrides an implicit deny, and there is no explicit Deny blocking EC2, her effective permissions will expand to successfully launch EC2 instances.

2. A new developer joins the team and they should have the same access as charlie-ops but also be allowed to upload files to S3. What is the cleanest way to configure this without modifying the group's policy?

The cleanest way is to add the new developer to the standard user group for baseline access, and then attach a personal inline policy directly to their user identity to grant the extra upload permission.

- charlie-ops represents the clean baseline state, inheriting standard permissions purely from group membership without any extra modifications.
- To mirror this cleanly, you should add the new developer to the same group (CloudOpsTeam), giving them the base S3 read-only and EC2 description capabilities.
- To give them upload capabilities without altering the rest of the team's access, you can follow the exact pattern used for Alice earlier in the lab: attach a personal identity policy directly to the new user that explicitly grants s3:PutObject.

3. A security incident occurs. You need to immediately ensure that no member of CloudOpsTeam can access S3 at all, including reads, without deleting any users or the group itself. What is the fastest way to do this?

The fastest way is to attach a new inline or managed policy directly to the CloudOpsTeam group containing an explicit Deny statement for all S3 actions (s3:*).

- **The Power of Explicit Deny:** As proven in Step 5 of the lab, an explicit Deny in any applicable policy always completely overrides and silences an Allow, regardless of where that allow comes from (group inheritance or personal identity policies).
- **Instant Blast Radius:** Attaching a policy like Deny on Action: "s3:*" at the group level will instantly block all members from reading, writing, or deleting anything in S3. This is identical to the real-world emergency offboarding/incident scenario noted in your documentation.

4. Bob's manager tells you to give Bob the ability to create EC2 instances. You cannot remove the permission boundary because company policy requires it. What are your options?

Because effective permissions require an action to be allowed by **both** the identity policy **and** the permissions boundary, you have only one real administrative option to fulfill this request:

- **Modify the existing Permission Boundary policy:** You must edit the boundary-s3-readonly-only policy (or update Bob to a new allowed boundary profile) to explicitly whitelist EC2 actions, specifically ec2:RunInstances and ec2:DescribeInstances.

Bob's identity policy already allows everything (Action: *), but he is blocked entirely because the boundary acts as a rigid ceiling filtering out anything outside of S3. Adding more permissions to his identity policy will do nothing. You *must* expand the boundary "box" itself to let the EC2 permissions through

Create a fourth user named devops-ops. This user must be able to launch EC2 instances and create S3 buckets, but must never be able to delete anything in S3 or IAM, even if future policies grant them that access. Implement this using the most appropriate mechanism and explain your choice.

The Solution: IAM Permissions Boundary

To satisfy all requirements perfectly—especially the condition that this user must **never** be able to delete anything even if future policies grant them that access, the most appropriate mechanism is an **IAM Permissions Boundary** combined with a standard identity-based policy.

1. The Strategy

- **Identity-Based Policy:** Grants the user the active permissions to perform `ec2:RunInstances` (launching instances) and `s3:CreateBucket` (creating buckets).
- **Permissions Boundary:** Acts as a rigid "ceiling" or filter. It will explicitly allow the required EC2 and S3 creation actions, but completely omit or explicitly deny any S3 or IAM deletion actions (`s3:Delete*` and `iam:Delete*`).

Because an action is only allowed if it is permitted by *both* the identity policy *and* the boundary, the boundary guarantees that future identity policies can never break through and grant delete capabilities.

2. Implementation Steps

Step A: Define the Identity-Based Policy

Attach a policy (either inline or managed) directly to devops-ops to grant their core operational capabilities:

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowEC2AndS3Creation",
      "Effect": "Allow",
      "Action": [
        "ec2:RunInstances",
```

```

        "ec2:Describe*",
        "s3:CreateBucket",
        "s3:ListAllMyBuckets",
        "s3:ListBucket"
    ],
    "Resource": "*"
}
]
}

```

Step B: Define and Attach the Permissions Boundary

Create a policy named devops-ops-boundary and attach it as the **Permissions Boundary** for the devops-ops user.

JSON

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowMaxCeiling",
      "Effect": "Allow",
      "Action": [
        "ec2:*",
        "s3:*",
        "iam:*"
      ],
      "Resource": "*"
    },
    {
      "Sid": "StrictDeleteBlockade",
      "Effect": "Deny",

```

```

    "Action": [
        "s3:Delete*",
        "iam:Delete*"
    ],
    "Resource": "*"
}
]
}

```

- **Why not just a standard Group or Inline policy?** If you only use standard identity policies (like you did for Alice) , a future administrator could easily attach a new policy that says Action: s3:DeleteObject or Action: *. That new Allow would immediately override the previous restriction because the absence of a permission is only an *implicit deny*.
- **Why the Boundary is the "Future-Proof" Choice:** A permissions boundary sets the absolute maximum ceiling for an identity. It says: *"You can only do what fits inside this box, and everything outside of it is structurally ignored"*. By explicitly putting a Deny for delete actions inside the boundary, it becomes an un-bypassable gate.
- **The Evaluation Formula Rule:** As demonstrated by Bob's behavior in Step 6, even if a future admin gives devops-ops an identity policy with Action: "*" (full administrator) , the IAM evaluation logic requires authorization from **both** sides. Because the deletion actions fail the boundary check, the request will be instantly blocked with an "Access Denied" banner every single time.